

Course Outline: Object Representation in Python

Title: Object Representation in Python **Skill:** Skill 13 — Relacionar objetos de una aplicación con su representación en tablas en bases de datos mediante la utilización de ORMs **Learnpack:** + Object Representation in Python **File:** object-representation-in-python **Prerequisite:** object-oriented-programming-in-python **Target Audience:** Beginner developers who know Python class syntax (class, `init`, `self`, methods) and are ready to model real-world entities with relationships between them **Total Lessons:** 8 **Format:** Mixed (25% hands-on = 2 lessons)

Course Description

You already know how to write a Python class. This course is about using that knowledge to accurately model real-world entities — entities that have relationships with other entities and belong to collections. By the end, you will be designing Python models whose structure mirrors reality closely enough to connect directly to a database: the foundation of Object-Relational Mapping.

Course Philosophy

This course emphasizes:

- Accuracy over simplicity: a model that misrepresents its domain will cause bugs that are hard to trace
 - Objects as building blocks: complex entities are built from simpler ones, not from strings
 - Practical readability: chained dot notation (`order.customer.email`) is clearer than string parsing
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Modeling Entities	3
02 - Collections	2
03 - Assessment	1
04 - Outro	1
Total	8

00.0 From Classes to Models

Learning Objectives:

- Distinguish between a class that stores data and a class that accurately models a real-world entity

- Understand what a "has-a" relationship means at a high level
- Know what this course covers and how it connects to ORM

Content Outline:

- The gap between knowing class syntax and designing good models
- What "modeling" means: does your class reflect reality, or just hold some fields?
- The two modeling challenges this course solves: (1) what to do when an attribute is itself a complex thing, (2) what to do when you have many instances of an entity
- Brief preview of Object-Relational Mapping as the destination: ORMs are just this, but connected to a database

Transitions: Section 01 starts by examining what makes a model accurate — specifically, how to choose attributes that represent an entity faithfully.

Key Principles:

- A class that models a real-world entity should reflect that entity's structure, not just its string representation
- The skills in this course are a direct prerequisite for ORM

Examples:

- A `User` class with `city = "Miami, FL, USA"` vs. `address = Address("Miami", "FL", "USA")` — which lets you do `user.address.city`?
- A restaurant app with a single `Restaurant` object vs. a list of `Restaurant` objects in a `City`

01.0 Modeling Entities

Learning Objectives:

- Explain what "accurate modeling" means in OOP
- Identify when a primitive attribute (string, int) is insufficient to represent an entity's property
- Describe a "has-a" relationship between two entities
- Preview what the next two lessons will cover

Content Outline:

- What is a model? A class whose attributes reflect what the entity actually is
- Two dimensions of good modeling: correct attribute types, correct relationships
- When string attributes are accurate enough (a name, a title, a status)
- When string attributes fail: anything that is itself a structured entity (an address, a payment method, a department)
- The "has-a" relationship: an `Employee` *has-a* `Department`; a `Restaurant` *has-a* `Address`
- This section: 01.1 deep-dives into object-typed attributes; 01.2 is a coding exercise

Transitions: Lesson 01.1 zooms into the specific technique for representing "has-a" relationships: using another class instance as an attribute.

Key Principles:

- If an attribute has its own attributes, it should be its own class
- "Has-a" (composition) is different from "is-a" (inheritance): a `Restaurant` has-an `Address`, it is not an `Address`
- Good models make downstream code simpler — accessing `restaurant.address.city` is cleaner than parsing a string

Examples:

- `Employee` with `department_name = "Engineering"` vs. `department = Department("Engineering", 3)` — the first loses the floor number and makes querying awkward
 - `Order` with `customer_email = "alice@example.com"` vs. `customer = Customer("Alice", "alice@example.com")` — the second gives access to the full customer object
-

01.1 One Object Inside Another

Learning Objectives:

- Define a class that holds another class instance as an attribute
- Access nested data using chained dot notation
- Recognize the "flat string" anti-pattern and explain its consequences

Content Outline:

- Defining two related classes: the "inner" class (e.g., `Department`) and the "outer" class (e.g., `Employee`)
- Passing an instance of the inner class as an argument to the outer class's `__init__`
- Accessing nested attributes: `emp.department.name`, `emp.department.floor`
- How deep chaining works and when it becomes a smell (more than 2-3 levels)
- Anti-pattern: flat string for structured data

Transitions: Now that you understand the pattern conceptually, lesson 01.2 gives you a coding challenge to build a nested model from scratch.

Key Principles:

- The inner class instance is just another attribute — `self.department = department` works exactly like `self.name = name`
- Chained dot notation is readable and precise: each `.` is a step into a related object
- An object attribute gives you full access to the inner object's methods as well, not just its data

Examples:

- `emp.department.name` → `"Engineering"`
- `emp.department.floor` → `3`
- Chained method call: `emp.department.get_budget()` if `Department` has such a method

Anti-pattern: Flat string for structured data

```
# Anti-pattern: packing structured data into a string
class Employee:
    def __init__(self, name, department_info):
        self.name = name
        self.department_info = department_info # "Engineering, Floor 3"

emp = Employee("Alice", "Engineering, Floor 3")
# Accessing the floor: emp.department_info.split(", ")[1] - fragile,
unreadable
```

Why it is tempting: it feels simpler to pass one string instead of creating a whole class.

What goes wrong: string parsing is brittle (what if the format changes?), you lose type safety, and you cannot call methods on the department.

The correct approach:

```
class Department:
    def __init__(self, name, floor):
        self.name = name
        self.floor = floor

class Employee:
    def __init__(self, name, department):
        self.name = name
        self.department = department

emp = Employee("Alice", Department("Engineering", 3))
print(emp.department.name) # Engineering
print(emp.department.floor) # 3
```

01.2 Build a Nested Model

Challenge Prompt: Complete the `Address` and `Restaurant` class constructors so the test code below runs and prints the expected output.

Solution Code:

```
class Address:
    def __init__(self, street, city):
        self.street = street
        self.city = city

class Restaurant:
```

```

def __init__(self, name, address):
    self.name = name
    self.address = address

restaurant = Restaurant("La Piazza", Address("123 Main St", "Miami"))
print(restaurant.name)           # La Piazza
print(restaurant.address.street) # 123 Main St
print(restaurant.address.city)   # Miami

```

Challenge Code:

```

class Address:
    def __init__(self, street, city):
        # store street and city as instance attributes
        pass

class Restaurant:
    def __init__(self, name, address):
        # store name and address as instance attributes
        pass

# Do not modify below this line
restaurant = Restaurant("La Piazza", Address("123 Main St", "Miami"))
print(restaurant.name)           # Expected: La Piazza
print(restaurant.address.street) # Expected: 123 Main St
print(restaurant.address.city)   # Expected: Miami

```

02.0 Collections of Objects

Learning Objectives:

- Explain why real applications work with lists of objects, not single instances
- Iterate over a list of object instances to access their attributes
- Use `max()` and `min()` with a `key` lambda to query an object collection
- Recognize the "parallel lists" anti-pattern and explain why it breaks

Content Outline:

- The real world has many instances: a company has many employees, a store has many products
- Storing object instances in a Python list: `employees = [Employee(...), Employee(...), ...]`
- Iterating: `for emp in employees: print(emp.name)`
- Querying with built-ins: `max(employees, key=lambda e: e.salary)` finds the highest-paid employee
- Anti-pattern: parallel lists instead of a list of objects
- Preview: lesson 02.1 is a coding exercise using these patterns

Transitions: Lesson 02.1 puts this into practice: you will build and query a collection of objects.

Key Principles:

- A list of object instances is the natural Python equivalent of a database table
- `key=lambda obj: obj.attribute` is the standard pattern for comparing objects by a field
- Every object in the list has the same structure — that consistency is what makes querying reliable

Examples:

- `max(products, key=lambda p: p.price)` → the most expensive product
- `[e for e in employees if e.salary > 70000]` → all well-paid employees
- `sorted(students, key=lambda s: s.grade, reverse=True)` → students ranked by grade

Anti-pattern: Parallel lists

```
# Anti-pattern: separate lists for related data
names = ["Alice", "Bob", "Carol"]
salaries = [75000, 62000, 88000]

# Finding the highest-paid employee:
max_index = salaries.index(max(salaries))
print(names[max_index]) # fragile – index alignment can break
```

Why it is tempting: it seems like less setup than defining a class.

What goes wrong: the two lists can become misaligned if one is sorted or filtered independently; there is no type safety; adding a third attribute (e.g., department) means a third list.

The correct approach:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

employees = [Employee("Alice", 75000), Employee("Bob", 62000),
Employee("Carol", 88000)]
top = max(employees, key=lambda e: e.salary)
print(top.name) # Carol – no index arithmetic needed
```

02.1 Manipulate a Collection

Challenge Prompt: Given the list of `Employee` objects below, find and print the name and salary of the highest-paid employee in the format `name: $salary`.

Solution Code:

```

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

employees = [
    Employee("Alice", 75000),
    Employee("Bob", 62000),
    Employee("Carol", 88000),
]

highest_paid = max(employees, key=lambda e: e.salary)
print(f"{highest_paid.name}: ${highest_paid.salary}")
# Carol: $88000

```

Challenge Code:

```

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

employees = [
    Employee("Alice", 75000),
    Employee("Bob", 62000),
    Employee("Carol", 88000),
]

# Find the employee with the highest salary and print:
# "name: $salary"
# your code here

```

03.0 What You Know Now 🧠

Learning Objectives:

- Identify when an attribute should be an object rather than a primitive
- Apply chained dot notation to access nested object data
- Recognize and explain the flat-string and parallel-lists anti-patterns
- Use `max()` / `min()` with a lambda to query a list of objects

Question Format: Scenario-based multiple choice

Questions:

1. A `Hotel` class stores its location. Which attribute design is better?

- ☐ `self.location = "Downtown, Miami, FL"`
- ☒ `self.location = Address(district="Downtown", city="Miami", state="FL")`
- ☐ `self.city = "Miami"` and `self.state = "FL"` as separate string attributes on `Hotel`
- ☐ `self.location = {"district": "Downtown", "city": "Miami"}`

Why the second option is best: the `Address` class is its own entity with its own attributes and potential methods. Separate string attributes on `Hotel` (option 3) pollute the class; a dictionary (option 4) loses type safety and dot-notation access.

2. Given `emp = Employee("Alice", Department("Engineering", 3))`, what does `emp.department.floor` return?

- ☐ `"Engineering"`
- ☐ An error — you cannot chain dot notation more than once
- ☒ `3`
- ☐ `Department("Engineering", 3)`

3. You have `names = ["Ana", "Luis"]` and `scores = [88, 72]` as two separate lists. What is the main risk?

- ☐ Python does not allow two lists with the same length
- ☒ Sorting or filtering one list without the other breaks the alignment between names and scores
- ☐ You cannot use `max()` on a plain list of integers
- ☐ There is no risk — parallel lists are the recommended pattern for related data

4. What is the correct way to find the `Product` with the lowest price from a list called `catalog`?

- ☐ `catalog.min()`
- ☐ `min(catalog)`
- ☒ `min(catalog, key=lambda p: p.price)`
- ☐ `catalog.sort()[0]`

5. A `Course` class has `self.instructor = "Dr. Smith, MIT, Computer Science"`. What is wrong?

- ☐ Nothing — storing a formatted string is the simplest approach
- ☒ The instructor is a structured entity; parsing the string later is fragile and hard to maintain
- ☐ You cannot store a string in a class attribute
- ☐ The string should use a dictionary instead

6. Which of the following correctly iterates over a list of `Student` objects and prints each name?

- ☐ `for s in students: print(students.name)`
- ☒ `for s in students: print(s.name)`
- ☐ `print(students[name])`
- ☐ `students.forEach(s => print(s.name))`

7. An `Order` class stores a customer. A colleague writes `order.customer_email = "alice@example.com"` instead of `order.customer = Customer(...)`. What future problem does this create?

- ☐ None — a string attribute is always sufficient for an email
- ☒ If you later need the customer's name or phone number, you must add more flat attributes to `Order`; a `Customer` object would already hold them
- ☐ Python does not allow string attributes on classes
- ☐ The email will not be accessible via dot notation

8. What does `max(employees, key=lambda e: e.salary)` return?

- ☐ The highest salary value as a number
- ☒ The `Employee` object with the highest salary
- ☐ A list of employees sorted by salary
- ☐ An error — `max()` only works on lists of numbers

Evaluation Criteria:

- 8/8: Excellent — ready for ORM
- 6–7: Good — review any missed questions before continuing
- 4–5: Review sections 01 and 02 before proceeding
- Below 4: Revisit both sections in full

Score Interpretation: The assessment covers all four course competencies: attribute design, nested objects, collections, and querying. Any question missed points to a specific lesson to revisit.

04.0 Course Outro

Content Outline:

- Course recap: what students accomplished
- Connection to bigger picture
- Next steps and continued learning
- Resources for further exploration
- Encouragement and celebration

Recap: You started this course knowing Python class syntax. You now know how to design classes that model real-world entities accurately: choosing object-typed attributes when an attribute is itself a complex entity, using chained dot notation to navigate relationships, and working with collections of object instances. These are not just Python skills — they are modeling skills.

Connection to bigger picture: Everything you practiced in this course is exactly what an Object-Relational Mapper (ORM) does — except that an ORM also connects each class to a database table and each attribute to a column. When you write `restaurant.address.city` in Python, an ORM translates that into a SQL JOIN and a column lookup. The mental model is identical; the ORM just handles the persistence layer.

Next steps: The next learnpack introduces ORM directly. You will see how the **Restaurant** and **Address** classes you built here become database-backed models with almost no changes to their structure.

Resources:

- SQLAlchemy documentation: <https://docs.sqlalchemy.org>
- 4Geeks ORM learnpack (next in this course sequence)

Note: This is conclusion only — no theory, exercises, or assessment.